

Trace as State: Reasoning Traces as Conditional States for Long-Context Transformers

Xu Zou¹ Jie Tang²

¹Z.ai ²Tsinghua University

Abstract

Transformers process information causally, but long-context reasoning may depend on task state discovered only later. We formalize this mismatch through conditional state update tasks. For causal state update processors, providing the condition first can require exponentially less memory in the worst case than providing it last.

Motivated by this principle, we introduce TRACE AS STATE. We use collected reasoning traces as a textual proxy for task state and place it before the long-context block on a fresh pass, allowing information derived previously to guide rereading.

We conduct extensive experiments on TRACE AS STATE and TRACE APPEND, a matched control that uses the same task state proxy but put it after the context. Across three models and three long-context datasets, TRACE AS STATE outperforms TRACE APPEND in 26 of 27 reported combinations of model, task, and metric. On GraphWalks Parents, exact match lifts DeepSeek V4 Pro(Preview) from 29.2% on the initial pass and 43.0% with TRACE APPEND to 81.8% with TRACE AS STATE, and from 66.4% and 83.2% to 100.0% for GLM-5.2. These results show that placing traces before the context can improve long-context reasoning while retaining the causal transformer structure.

1 Introduction

Frontier language models can now accept inputs extending to hundreds of thousands or even millions of tokens. Advances in sparse, compressed, and hybrid sequence modeling architectures have substantially enlarged their nominal context windows (DeepSeek-AI, 2025b, 2026a; Qwen Team, 2025a; Yang et al., 2025).

Despite these architectural advances, autoregressive Transformers retain causal attention. Information appearing later remains available to subsequent

reasoning and answer generation, but it cannot affect the representations formed at earlier positions within the same pass.

One structural difficulty is a mismatch between input order and reasoning order. Prior work shows that reasoning performance can depend not only on where supporting information appears, but also on the relative order in which it is presented (Chen et al., 2024; Yu et al., 2025). In many long context tasks, a solver must maintain task-relevant state, such as an active target, a search frontier, or a set of rejected hypotheses. Some of this state may become available only after earlier parts of the context have already been processed.

We analyze this asymmetry through conditional state update tasks. Such a task begins from a state specified by a condition and applies an information sequence to it. If the condition is available before the sequence, a causal processor can update the realized state as each item arrives. If the condition arrives after the sequence, the processor may instead need to retain how the sequence would act on every possible condition. We show that these two input orders can have exponentially different memory requirements in the worst case. The separation motivates an ordering principle for causal transformer passes: task-relevant state discovered late in one pass can be made available before the context in the next pass. Figure 1A and 1B illustrate the causal processor and the two input orders.

Modern reasoning models provide an observable interface through the reasoning traces they generate before producing visible answers (DeepSeek-AI, 2025a; Qwen Team, 2025b; Lee et al., 2025). Generated reasoning text can carry intermediate computational information across steps (Nye et al., 2021; Wei et al., 2022; Merrill and Sabharwal, 2024; Levy et al., 2025). Reasoning traces may be incomplete or contain errors, and we treat them as an observable textual proxy for task state, as shown in figure 1C. We therefore introduce TRACE AS STATE,

a general inference scaling method that follows a read, compute, feedback, and reread procedure. The model first processes the task and generates one or more reasoning traces. We serialize the collected traces as T and place it before the long-context block in a fresh pass.

We compare TRACE AS STATE with TRACE APPEND, a placement control that gives the second pass serialized trace text after the long context. Under TRACE AS STATE, the trace is available while the context is processed again. Under TRACE APPEND, the trace can still guide later reasoning and answer generation, but it cannot influence the representations already formed for the preceding context. Figure 1D summarizes the strict placement design.

We evaluate this placement intervention on GraphWalks 256K, MRCRv2 8-needle, and NUB-1M using three frontier models from different providers: DeepSeek V4 Pro Preview, GLM-5.2, and Qwen 3.7 Max. Across the 27 reported combinations of model, task, and metric, TRACE AS STATE outperforms TRACE APPEND in 26. The result shows that the benefit of using reasoning traces as a task state proxy and placing it before the context is general in long context reasoning, and is consistent with our conditional state update task memory analysis.

We further test alternative explanations through ablations on GraphWalks 256K using DeepSeek V4 Pro Preview. These controls weaken explanations based solely on generic trace scaffolding or access to first-pass answers and further support the TRACE AS STATE interpretation.

Our contributions are as follows:

- We use a conditional state update abstraction to characterize a worst-case exponential order separation for deterministic processors that read the input once, and apply it as a qualitative principle for causal processing of long contexts.
- We introduce TRACE AS STATE, a general inference scaling method for reasoning over long contexts that feeds reasoning traces back as textual state proxies. The method adds feedback between passes while retaining causal processing within each pass.
- Across three models and three benchmarks for reasoning over long contexts, TRACE AS STATE scores above TRACE APPEND in 26 of

27 reported combinations of model, task, and metric.

2 Related Work

Long-context architectures and effective use.

Sparse, compressed, and hybrid sequence models have extended the nominal context windows of language models (DeepSeek-AI, 2025b, 2026a; Qwen Team, 2025a; Yang et al., 2025). Long-context evaluations nevertheless show continued sensitivity to evidence position, distractors, and the operations required to combine information across an input (Liu et al., 2024; Kuratov et al., 2024; Hsieh et al., 2024).

Causal order in reasoning.

Under causal attention, a representation at one position can use only information from that position and its prefix. Ok and Lee attribute a large multi-choice prompt-order gap to this constraint and show that repeating the options after the context partially closes the gap (Ok and Lee, 2026). CoRe reduces sensitivity to the order of supporting documents by repeating the full context, whereas *Racing Thoughts* traces contextualization errors to layerwise race conditions (Yu et al., 2025; Lepori et al., 2025). Collectively, these studies show that input order, repetition, and contextualization can affect model behavior.

Architectural recurrence.

Iterative language models can revisit hidden representations or partially specified text. Geiping et al. train a language model with recurrent depth that repeatedly applies a shared block at test time (Geiping et al., 2025). Saunshi et al. study Transformer blocks with shared weights, and *LoopFormer* trains variable loop trajectories for different inference budgets (Saunshi et al., 2025; Jeddi et al., 2026). Masked text-diffusion models such as LLaDA reconstruct masked positions over multiple steps using visible context on both sides (Sahoo et al., 2024; Nie et al., 2025). These approaches expand the design space for iterative computation. However, they can require substantial changes to existing training and inference infrastructure.

Rereading and textual feedback.

Several methods revisit task information or carry textual state forward during inference. Re2 repeats the question within one prompt and provides a direct rereading baseline without prior reasoning text (Xu et al., 2024). The Markovian Thinker carries a bounded text history across reset reasoning chunks, while

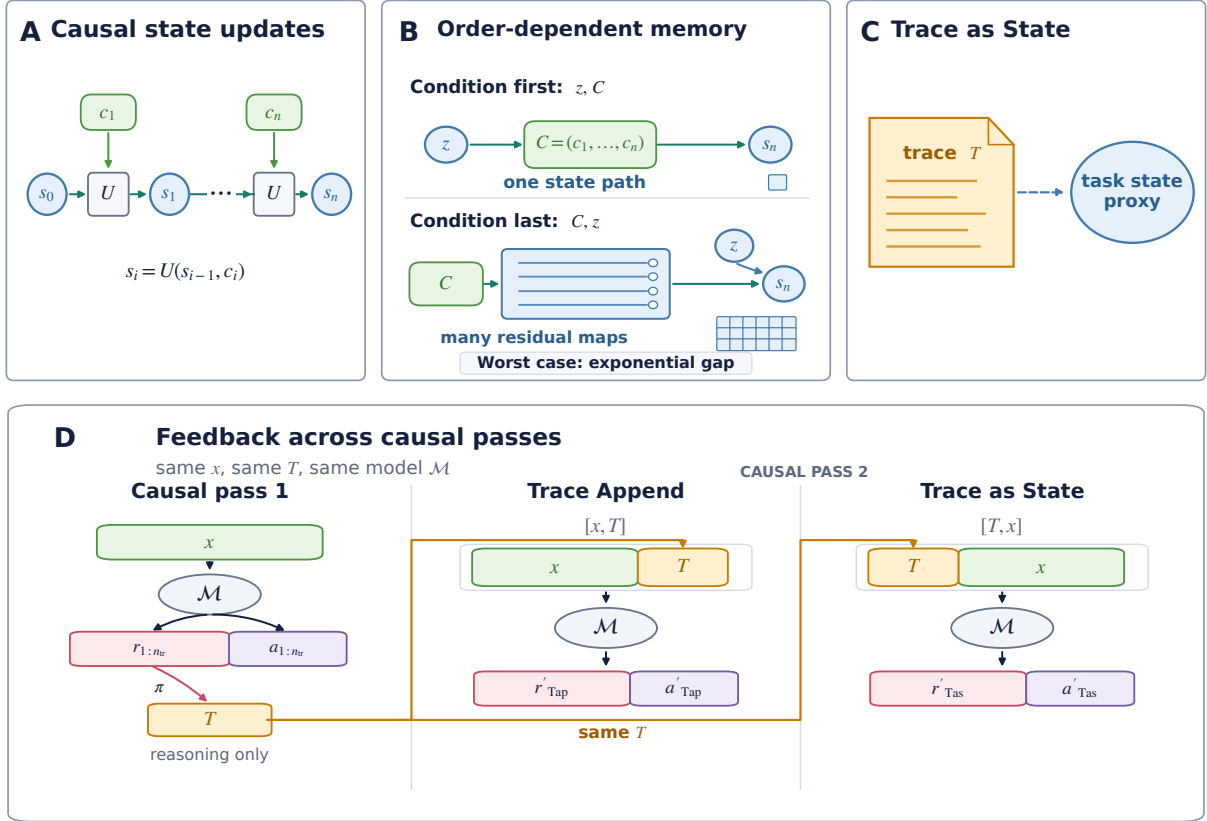


Figure 1: **Overview for TRACE AS STATE** (A) A causal processor carries one running state. (B) In a worst-case conditional state update task, condition-first processing tracks only the realized state, whereas condition-last processing can require exponentially more working memory. (C) Reasoning trace T provides a textual proxy for task state. (D) TRACE AS STATE places T before the context in a fresh causal pass.

ReContext recursively builds and replays an evidence pool for the current query (Aghajohari et al., 2026; Zhao et al., 2026). These methods show that feeding task-relevant text back to a model can improve reasoning.

Reasoning traces as state. Reasoning traces can serve not only as explanations, but also as textual records of an evolving computational state. Hao et al. show that synthetic hints inserted into a reasoning trace can affect later outputs even when follow-up explanations do not acknowledge their influence (Hao et al., 2026). The *State over Tokens* preprint describes the growing reasoning prefix as externalized computational state, whereas causal mediation evidence suggests that models do not reliably use their stated intermediate steps (Levy et al., 2025; Paul et al., 2024). Taken together, the evidence shows that reasoning traces can carry task-relevant information and affect later outputs.

3 Methodology

Textual reasoning typically follows a causal order, which is well matched by causal transformers. Yet some reasoning depends on task states whose values become known only later, creating a mismatch especially consequential in long contexts. We formalize this mismatch using causal state update systems and conditional state update tasks and introduce TRACE AS STATE.

Table 1 summarizes the vital notation used in this section.

3.1 Causal State Updates and Order-Dependent Memory

Causal state updates. We define a causal state update processor as a processor that reads each input unit once in its presented order and updates its persistent working memory using only its current memory and the newly received unit.

Let $C = (c_1, \dots, c_n)$ be an ordered sequence of n information units, with c_i denoting the i th unit. As shown in Figure 1A, a causal state update

Table 1: Vital Notations.

Symbol	Meaning
c_i, C	The i th information unit and the ordered sequence $C = (c_1, \dots, c_n)$.
$s_i, \mathcal{S}, b$	The task state after c_i , its finite state space, and the log size of the state space
U	The causal state update rule.
z	The condition used as the initial task state s_0 .
$x, \mathcal{M}$	The long context and the causal reasoning model.
r_j, a_j, n_{tr}	The reasoning trace, visible answer, and number of source model runs.
π, T	The trace serializer and the resulting serialized trace.

processor carries a task state s_i in a finite state space $\mathcal{S}$. After reading c_i , it applies a fixed update rule U :

$$s_i = U(s_{i-1}, c_i), \quad i = 1, \dots, n. \quad (1)$$

The state s_i is the task state the processor reached after processing $c_1, \dots, c_i$. When the initial state s_0 is fixed, the processor follows one realized state path through the sequence.

Conditional state update tasks and order-dependent memory. Consider a variant of the causal state update process in which the initial state is supplied by a task condition rather than being fixed. Let $z \in \mathcal{S}$ denote this condition. We call the resulting problem a conditional state update task: the processor sets $s_0 = z$, applies Equation 1 to C , and returns s_n .

Figure 1B compares two orders for the same condition and information sequence. In the condition first order $[z, C]$, the processor receives z before $c_1, \dots, c_n$ (n is the length of the information sequence) and therefore knows which state path to update. In the condition last order $[C, z]$, it reads the complete sequence before learning which initial state should be propagated through it.

To compare their working memory, consider a causal state update processor that knows exactly the state update rule U . With $[z, C]$, the processor only needs to store the current s_i , so $\lceil b \rceil$ bits suffice, where $b = \log_2 |\mathcal{S}|$.

With $[C, z]$, the processor has not selected a state path when it finishes reading C . Each possible sequence determines a complete response profile: for every $z \in \mathcal{S}$, the profile specifies the resulting s_n . The processor must retain a different configuration for every distinct response profile induced by the valid sequences.

There are $|\mathcal{S}|^{|\mathcal{S}|}$ possible functions from $\mathcal{S}$ to itself. In the worst case, $[C, z]$ requires at least

$$\lceil \log_2 |\mathcal{S}|^{|\mathcal{S}|} \rceil = \lceil |\mathcal{S}| \log_2 |\mathcal{S}| \rceil = \lceil b 2^b \rceil \quad (2)$$

bits, whereas $[z, C]$ requires only $\lceil b \rceil$ bits. In the worst-case scenario, the memory requirement is exponentially larger in the condition last setting than in the condition first setting.

We could derive an ordering principle: a task condition could be much easier to use when it is available before the information whose processing it guides.

3.2 Reasoning Traces as a Textual State Proxy

Although transformers may retain per-token kv caches, causal transformers with finite context length and finite precision are causal state update models as the their max memory are bounded. Conditional state update tasks are not literal models of real world long context tasks. Instead, the results in section 3.1 motivate us to test placing task relevant state before the context on a later pass.

The formal condition z represents task state that is already available to the processor. In a long context reasoning problem, however, useful task state may be discovered only while the model is reasoning after reading the context. A reasoning trace may record useful task states like an active target, a resolved reference or a search frontier.

We view reasoning traces as an observable textual proxy for task state, as illustrated in figure 1C. The proxy may be incomplete, lossy, or incorrect. We therefore do not identify a trace with the formal condition z or with a privileged internal model state. The connection is functional: if the trace contains useful state information, placing it before the context can make that information available while the context is processed again.

Let $\mathcal{M}$ be a causal state update model and let x denote the long context. We run $\mathcal{M}$ n_{tr} times on the same problem. Each run j produces a reasoning trace r_j and a separate visible answer a_j :

$$(r_j, a_j) \sim \mathcal{M}(\cdot \mid x), \quad j = 1, \dots, n_{\text{tr}}. \quad (3)$$

For each task, a serializer π held fixed across the placement conditions constructs the serialized trace

$$T = \pi(r_1, \dots, r_{n_{\text{tr}}}). \quad (4)$$

The serializer preserves the included reasoning text in source order and adds fixed labels and delimiters. We use T as the textual state proxy to be tested.

3.3 Trace as state.

Figure 1D shows the complete procedure. Based on theoretical analysis from section 3.1, we introduce TRACE AS STATE, a method that places the textual state proxy T before the long context x . Therefore, in TRACE AS STATE, a fresh causal pass receives $[T, x]$ to generate the answer again. We compare TRACE AS STATE with TRACE APPEND, the method that maintains the original order of x and T and sends the model $[x, T]$ on a second pass.

Let r' and a' denote the reasoning trace and visible answer produced in this pass:

$$\begin{aligned} (r'_{\text{Tap}}, a'_{\text{Tap}}) &\sim \mathcal{M}(\cdot \mid [x, T]), \\ (r'_{\text{Tas}}, a'_{\text{Tas}}) &\sim \mathcal{M}(\cdot \mid [T, x]). \end{aligned} \quad (5)$$

TRACE AS STATE and TRACE APPEND use the same long context x and the same textual task state proxy T , with order as the only difference. With TRACE AS STATE, T is available while the model processes x again. With TRACE APPEND, T arrives after x and cannot change representations already formed for the preceding context tokens, although it can still influence subsequent reasoning and the visible answer. The comparison therefore tests whether the same T is more useful during rereading than after the long context has already been processed.

4 Experiments

4.1 Setup

In this section, we evaluate TRACE AS STATE and its matched placement control TRACE APPEND on long context tasks from different domains. The models come from different providers and have different disclosed architectural designs. A fixed serializer preserves the reasoning traces while adding only necessary delimiters such as "`<trace_start>`" and "`<trace_end>`" and brief introductory text to construct T . Some traces are so long that we truncate them to the first 50,000 characters to keep the second-pass prompt within the model’s context capacity. These choices define the realization evaluated here; the general TRACE AS STATE framework also permits adapted models, serializers, or state interfaces while retaining causal processing within each pass.

Models. We evaluate Qwen 3.7 Max, DeepSeek V4 Pro Preview, and GLM-5.2. All three are frontier long-context reasoning models that expose reasoning traces that can be logged and supplied back to the model as an imperfect textual

proxy that may carry task-state information. They also represent different providers and disclosed long-context designs: DeepSeek V4 Pro Preview uses a hybrid attention stack with Compressed Sparse Attention(CSA), Heavily Compressed Attention(HCA), and Manifold-Constrained Hyper-Connections(mHC) (DeepSeek-AI, 2026a); Qwen 3.7 Max is formed with Gated Deltanet(GDN) and Gated Attention(GA) (Yang et al., 2025; Qiu et al., 2025); and GLM-5.2 is a 1M-token MoE model with DeepSeek Sparse Attention (DSA) optimized by IndexCache (IC), which reuses sparse-attention indices across layers (DeepSeek-AI, 2025b; Bai et al., 2026; GLM-5 Team, 2026; Z.ai, 2026b,a). We choose the highest available reasoning effort in our experiments: `max` for DeepSeek V4 Pro and GLM-5.2. For Qwen 3.7 Max, we use the official `xhigh` system prompt. All three support approximately one-million-token inputs in the evaluated interfaces and expose the reasoning traces required by the frozen textual realization evaluated here (Alibaba Cloud, 2026; DeepSeek-AI, 2026b; Z.ai, 2026a).

Details of the models are described in Table 1-1.

Datasets. We use GraphWalks (OpenAI, 2025a), MRCRv2 8-needle (OpenAI, 2025b; Vodrahalli et al., 2024), and 1M-Novel Understanding Bench (NUB-1M) (xz-keg, 2026) for our evaluation. GraphWalks requires the model to maintain graph state over a long edge list; MRCRv2 asks the model to bind a final request to the correct earlier request-response instance. Both provide multiple prompt-length bins. As different models use different tokenizers, many problems in the 1M bin cannot be fairly tested due to overlength. We therefore choose the longest bins under 1M: The 256K bin for GraphWalks and the 256K and 512K bins for MRCRv2. This choice also leaves room for additional TRACE AS STATE and TRACE APPEND texts.

NUB-1M is a long novel reading comprehension benchmark with complex questions about a new novel containing 400–700K tokens. The dataset is updated by season to reduce leakage risk. For each season, the problems and answers are manually maintained. We use the season 2 novel for evaluation and report average accuracy of the 20 problems over 5 repeats.

Models may not behave as intended unless the question appears at the end of the prompt. We therefore separate the question from the long context and place it at the end of every input to ensure

Table 1-1: Model configuration and token budgets used in the experiments.

Model	Qwen 3.7 Max	DeepSeek V4 Pro Preview	GLM-5.2
Arch. ^a	GDN+GA	CSA+HCA + mHC	DSA+IC
Input Budget	983,616	1,048,576	1,048,576
Output Budget	65,536	131,072	65,536
Reasoning	xhigh ^b	max	max

^a Architecture. ^b Via xhigh prompt.

Table 1-2: Datasets and scoring used in the experiments.

Task	GraphWalks	MRCRv2	NUB-1M
		256K, 512K	
Subsets	256K	8-needle	Season 2
Problems	200	200	20
Repeats	5	5	5
Scoring	EM ^a , set F1	EM, Seq. ^b	Acc. ^c

^a Exact Match. ^b SequenceMatcher ratio. ^c Accuracy.

models behave well focused on the given tasks. The literal orders are therefore TRACE AS STATE $[T, x, q]$ and TRACE APPEND $[x, T, q]$, where x is the long context and q is the question.

Details of the datasets are listed in Table 1-2.

For each model and task, we compare a single-pass baseline $\mathcal{M}([x, q])$ with two trace-backed conditions TRACE AS STATE, $\mathcal{M}([T, x, q])$ and TRACE APPEND $\mathcal{M}([x, T, q])$ that reuse the model’s own first-pass reasoning traces. Appendix C gives the trace serialization and dataset-specific insertion boundaries.

We evaluate each problem with 5 repeats and include all 5 reasoning traces in the second-pass TRACE AS STATE and TRACE APPEND prompts. We use the official provider for these models, Aliyun Bailian for Qwen 3.7 Max (Alibaba Cloud, 2026), DeepSeek for DeepSeek V4 Pro (DeepSeek-AI, 2026b), and Bigmodel for GLM-5.2 (Z.ai, 2026a). We do not explicitly pass a custom maximum-output value. The run records therefore establish that no client-side cap was requested, but not the effective provider default, which may also change over time.

Our evaluations use EM and set F1 for GraphWalks, EM and SequenceMatcher ratio for MRCRv2 8-needle, and DeepSeek V4 Pro model-

judged accuracy for NUB-1M. Blocked, overlong, malformed, missing, nonterminal, or content-filtered cases are scored as failures.

Some first pass reasoning traces are very long, we therefore truncate each reused trace to its first 50,000 characters to avoid input overlength.

4.2 Main Results

Table 2 displays the main experimental results. Across all 27 reported combinations of model, task, and metric, TRACE AS STATE scores higher than TRACE APPEND in 26. DeepSeek V4 Pro and Qwen 3.7 Max favor TRACE AS STATE on every reported dataset, task and metric. GLM-5.2 follows the same pattern except on GraphWalks BFS F1, where TRACE APPEND is 0.8 points higher while TRACE AS STATE has higher exact match. TRACE AS STATE also scores above the first pass in all reported evaluations.

TRACE APPEND improves over the first pass in many settings, showing that the trace text can carry useful information in these settings. The additional advantage of TRACE AS STATE is consistent with the hypothesis that placing reasoning traces as an imperfect textual task state proxy before the context is beneficial when the model processes the context again.

The strongest gains occur on GraphWalks Parents, where success requires the model to maintain predecessor state while interpreting the graph. DeepSeek V4 Pro improves from 46.5 to 91.3 F1, or 29.2 to 81.8 EM, and Qwen 3.7 Max improves from 87.2 to 99.1 F1, or 60.8 to 96.4 EM. GLM-5.2 reaches 100.0 EM and F1 on Parents under TRACE AS STATE. MRCRv2 retrieval and binding comparisons show the same ordering advantage. NUB-1M runs provide supporting evidence in the same direction for detailed long-novel reading comprehension.

4.3 Context Order Ablations

We next compare TRACE AS STATE with controls that vary how different context are placed and first-pass outputs are reused on DeepSeek V4 Pro GraphWalks 256K in table 3. We test several ablations to examine how models behave under different trace placements and feedback controls. For each ablation, the table labels the prompt format and reports the exact match(EM) and F1 score. We also include Majority@5 and Oracle@5 for the first pass in the table. Majority@5 evaluates the major choices of

Table 2: Long-context results for first pass, TRACE AS STATE and TRACE APPEND. Scores are percentages averaged over 5 repeats; higher is better. The best score within each model-metric column is **bolded**.

Model	Condition	GraphWalks 256K				MRCRv2 8-needle				NUB-1M
		BFS		Parents		256K		512K		Season 2
		EM	F1	EM	F1	EM	Seq.	EM	Seq.	Acc.
DeepSeek V4 Pro Preview	First Pass	31.6	36.4	29.2	46.5	53.8	78.5	45.4	63.6	60.0
	TRACE APPEND	41.8	48.3	43.0	65.3	66.6	79.6	49.4	66.9	71.0
	TRACE AS STATE	58.8	65.9	81.8	91.3	76.8	88.7	52.6	73.3	73.0
Qwen 3.7 Max	First Pass	60.0	68.1	60.8	87.2	79.8	83.1	36.2	43.7	37.0
	TRACE APPEND	60.4	70.4	71.0	91.7	84.0	87.1	40.4	47.4	49.0
	TRACE AS STATE	63.8	71.7	96.4	99.1	88.4	91.3	46.8	52.5	51.0
GLM-5.2	First Pass	55.8	70.7	66.4	88.5	40.2	48.2	42.6	52.0	43.0
	TRACE APPEND	60.0	75.8	83.2	92.7	40.0	58.3	44.6	59.9	65.0
	TRACE AS STATE	63.4	75.0	100.0	100.0	61.2	71.9	55.4	70.1	66.0

the 5 first pass answers, while Oracle@5 evaluates the best answer among the 5 answers.

Question First puts the question q before the task. It substantially improves performance on Parents but remains similar to the first pass baseline on BFS, implying that the question itself may be a vital task state for some tasks.

Re2 (Xu et al., 2024) improves substantially over the first pass, showing that rereading is useful on these long context tasks. However, it remains below TRACE AS STATE, especially on Parents. In this evaluated setting, the comparison is consistent with T carrying task-relevant information beyond that supplied by prompt repetition alone.

Answer Feedback places the first pass answers $a = [a_1, \dots, a_{n_{tr}}]$ before the original prompt. It improves over the first pass but remains far below TRACE AS STATE. This supports the interpretation that serialized reasoning text can serve as a more useful task state proxy than the answers or the questions alone.

Random Trace replaces the prefix with traces sampled from other problems in the same GraphWalks subtask. It performs worse than the no-trace first pass, much worse than TRACE AS STATE. So the gain of TRACE AS STATE is not due to a generic formatting effect, and a reasoning-trace-like scaffold alone is insufficient without problem-specific, task-relevant information from reasoning traces of the same problem.

All above controls above place a copy of the long context late in the prompt but remain below TRACE

AS STATE, signifying that textual recency alone does not explain the observed TRACE AS STATE gains.

Trace Only sends the first-pass reasoning traces without the original long context prompt. Its performance is similar to TRACE APPEND, showing that the traces contain useful answer-relevant information. Nevertheless, it remains well below TRACE AS STATE, showing that the original input is still useful when placed after the first pass traces. Trace as State also outperforms Oracle@5, showing that the feedback pass improves on what can be obtained by retrospectively selecting the best of the five first pass outputs.

4.4 Trace Count Ablation

Finally, we rerun DeepSeek V4 Pro Preview on the GraphWalks 256K while varying the number of reused first-pass traces from $n_{tr} = 1$ to $n_{tr} = 5$. The block for each count contains the first n_{tr} eligible traces in repeat order, so successive settings are nested. For every $n_{tr} \geq 1$, TRACE AS STATE and TRACE APPEND receive the same realized T , and each condition averages five fresh second-pass repeats per problem. The $n_{tr} = 0$ point is the common five-repeat first-pass mean. Figure 2 shows that performance generally rises as additional traces are included, and TRACE AS STATE remains above TRACE APPEND for every $n_{tr} \geq 1$. Within this frozen-model realization, the result supports trace count as an inference-scaling parameter and shows that the measured TRACE

Table 3: DeepSeek V4 Pro Preview performance on GraphWalks 256K under different prompt conditions. Scores are percentages averaged over 5 repeats. The best scores for each subtask/metric are **bolded**.

Condition	Prompt	BFS		Parents	
		Exact Match(%)	F1 Score(%)	Exact Match(%)	F1 Score(%)
First pass	$[x, q]$	31.6	36.4	29.2	46.5
Majority@5		35.0	39.6	31.0	47.8
Oracle@5		50.0	55.5	50.0	75.0
Question First	$[q, x, q]$	33.4	36.8	53.0	64.6
Re2 (Xu et al., 2024)	$[x, q, x, q]$	49.0	52.9	50.0	68.9
Answer Feedback	$[a, x, q]$	35.4	39.4	45.4	58.0
Random Trace	$[T_{\text{rand}}, x, q]$	22.4	35.9	14.2	31.3
Trace Only	$[T, q]$	46.2	52.7	43.8	67.3
TRACE APPEND	$[x, T, q]$	41.8	48.3	43.0	65.3
TRACE AS STATE	$[T, x, q]$	58.8	65.9	81.8	91.3

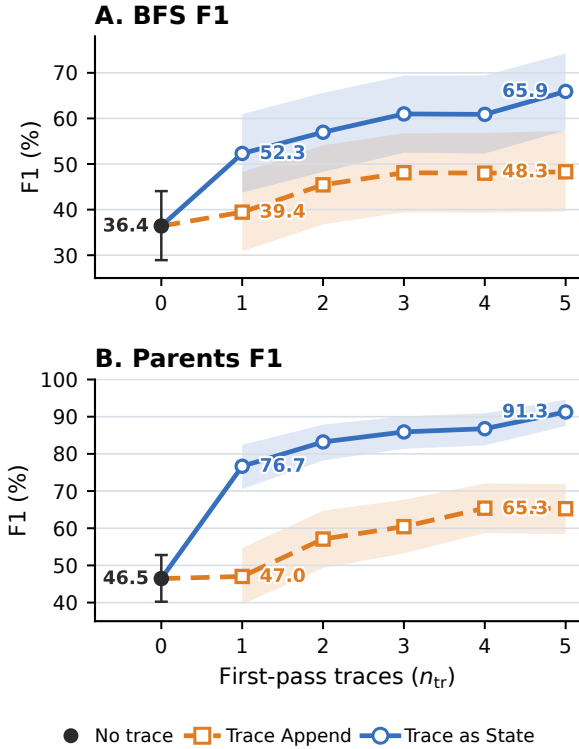


Figure 2: Trace-count ablation on DeepSeek V4 Pro GraphWalks 256K. Panel A reports BFS F1 and Panel B reports Parents F1. Shading shows 95% percentile confidence intervals.

AS STATE–TRACE APPEND ordering persists from $n_{tr} = 1$ through $n_{tr} = 5$.

5 Conclusion

We introduced TRACE AS STATE, an inference approach that reuses task state information carried

in reasoning traces as a textual proxy on a fresh pass over a long context task. Its motivation comes from theoretical analysis of conditional state update tasks: for causal state update processors, a condition available before an information sequence can guide a single evolving state, while a late condition can require retaining much more memory about potential conditions of the sequence.

TRACE AS STATE applies this ordering principle by making prior reasoning available before the long context block is processed again. Together, the formal analysis and experiments support a simple view: reasoning traces can carry forward task state information, and the point at which that information becomes available can shape how useful it is.

This view suggests several direct extensions: selecting or compressing traces, learning better textual state interfaces, and optimizing where feedback is placed. A broader training framework could jointly learn the model and the feedback interface while retaining causal processing within each pass.

Limitations

Our experiments realize cross-pass state feedback through model-generated text. This evaluated realization requires access to raw reasoning traces or another exposed state interface. The requirement comes from the interface used in our experiments. The general cross-pass design can instead use any accessible state interface. Models or APIs that expose only final answers may therefore require a different interface.

TRACE AS STATE uses one or more source runs followed by a fresh pass over the task. These additional passes increase inference latency and token cost. Placing state before the original context can also reduce key-value cache reuse in multi-round settings.

Our evaluation covers three causal transformer models from different providers and three long context task families: GraphWalks, MRCR, and NUB-1M. We do not evaluate multi-round agent tasks. Testing additional models, domains, context lengths, and interactive settings is needed to establish how broadly the observed placement advantage generalizes.

Finally, because the method reuses only model generated traces, it does not introduce additional ethical concerns or misuse risks beyond those associated with the underlying models and tasks.

References

- Milad Aghajohari, Kamran Chitsaz, Amirhossein Kazemnejad, Sarath Chandar, Alessandro Sordoni, Aaron Courville, and Siva Reddy. 2026. [The markovian thinker: Architecture-agnostic linear scaling of reasoning](#). In *The Fourteenth International Conference on Learning Representations*. Poster.
- Alibaba Cloud. 2026. qwen3.7-max model information. <https://help.aliyun.com/zh/model-studio/qwen3-7-max>. Official Model Studio documentation. Accessed August 3, 2026.
- Yushi Bai, Qian Dong, Ting Jiang, Xin Lv, Zhengxiao Du, Aohan Zeng, Jie Tang, and Juanzi Li. 2026. [IndexCache: Accelerating sparse attention via cross-layer index reuse](#). In *Third Conference on Language Modeling*.
- Xinyun Chen, Ryan Andrew Chi, Xuezhi Wang, and Denny Zhou. 2024. [Premise order matters in reasoning with large language models](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 6596–6620. PMLR.
- DeepSeek-AI. 2025a. [DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning](#). *Nature*, 645:633–638.
- DeepSeek-AI. 2025b. [DeepSeek-V3.2: Pushing the frontier of open large language models](#). Technical report arXiv:2512.02556, DeepSeek-AI.
- DeepSeek-AI. 2026a. [DeepSeek-V4: Towards highly efficient million-token context intelligence](#). Technical report arXiv:2606.19348, DeepSeek-AI.
- DeepSeek-AI. 2026b. Models & pricing. https://api-docs.deepseek.com/quick_start/pricing/. Official API documentation. Accessed August 3, 2026.
- Jonas Geiping, Sean Michael McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- GLM-5 Team. 2026. [GLM-5: From vibe coding to agentic engineering](#). Technical report arXiv:2602.15763, Z.ai and Tsinghua University.
- Yijie Hao, Lingjie Chen, Ali Emami, and Joyce C. Ho. 2026. [Reasoning traces shape outputs but models won’t say so](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 42852–42878, San Diego, California, United States. Association for Computational Linguistics.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. [RULER: What’s the real context size of your long-context language models?](#) In *First Conference on Language Modeling*.
- Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. 2026. LoopFormer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *The Fourteenth International Conference on Learning Representations*.
- Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. 2024. [BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack](#). In *Advances in Neural Information Processing Systems*, volume 37. Datasets and Benchmarks Track.
- Celine Lee, Alexander M. Rush, and Keyon Vafa. 2025. [Critical thinking: Which kinds of complexity govern optimal reasoning length?](#) In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*.
- Michael A. Lepori, Michael Curtis Mozer, and Asma Ghandeharioun. 2025. [Racing thoughts: Explaining contextualization errors in large language models](#). In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3020–3036, Albuquerque, New Mexico. Association for Computational Linguistics.
- Mosh Levy, Zohar Elyoseph, Shauli Ravfogel, and Yoav Goldberg. 2025. [State over tokens: Characterizing the role of reasoning tokens](#). *Preprint*, arXiv:2512.12777.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. [Lost in the middle: How language models use long contexts](#). *Transactions of the Association for Computational Linguistics*, 12:157–173.
- William Merrill and Ashish Sabharwal. 2024. The expressive power of transformers with chain of thought. In *The Twelfth International Conference on Learning Representations*.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. 2025. [Large language diffusion models](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 50608–50646. Curran Associates, Inc.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. 2021. Show your work: Scratchpads for intermediate computation with language models. ArXiv:2112.00114.

- Hyunjong Ok and Jaeho Lee. 2026. [Lost in the prompt order: Revealing the limitations of causal attention in language models](#). In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 38566–38587, San Diego, California, United States. Association for Computational Linguistics.
- OpenAI. 2025a. GraphWalks: A multi hop reasoning long context benchmark. <https://huggingface.co/datasets/openai/graphwalks>. Dataset card. Accessed August 3, 2026.
- OpenAI. 2025b. OpenAI MRCR: Long context multiple needle in a haystack benchmark. <https://huggingface.co/datasets/openai/mrcr>. Dataset card. Accessed August 3, 2026.
- Debjit Paul, Robert West, Antoine Bosselut, and Boi Faltings. 2024. [Making reasoning matter: Measuring and improving faithfulness of chain-of-thought reasoning](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15012–15032, Miami, Florida, USA. Association for Computational Linguistics.
- Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. [Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 110931–110957.
- Qwen Team. 2025a. [Qwen3-Next-80B-A3B-Instruct model card](#). Model card, Alibaba Cloud. Official model card.
- Qwen Team. 2025b. [Qwen3 technical report](#). Technical report arXiv:2505.09388, Alibaba Cloud.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. 2024. [Simple and effective masked diffusion language models](#). In *Advances in Neural Information Processing Systems*, volume 37.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. 2025. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*.
- Kiran Vodrahalli, Santiago Ontanon, Nilesch Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shivanna, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, Rohan Anil, Ethan Dyer, Siamak Shakeri, Roopali Vij, Harsh Mehta, Vinay Ramasesh, Quoc Le, Ed Chi, Yifeng Lu, and 5 others. 2024. [Michelangelo: Long context evaluations beyond haystacks via latent structure queries](#). Preprint, arXiv:2409.12640.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837.
- Xiaohan Xu, Chongyang Tao, Tao Shen, Can Xu, Hongbo Xu, Guodong Long, Jian-Guang Lou, and Shuai Ma. 2024. [Re-reading improves reasoning in large language models](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 15549–15575, Miami, Florida, USA. Association for Computational Linguistics.
- xz-keg. 2026. 1M Novel Understanding Bench. <https://github.com/xz-keg/Novel-Understanding-Bench>. Project repository. Accessed August 3, 2026.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2025. Gated delta networks: Improving Mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*.
- Sangwon Yu, Ik hwan Kim, Jongyoon Song, Saehyung Lee, Junsung Park, and Sungroh Yoon. 2025. [Unleashing multi-hop reasoning potential in large language models through repetition of misordered context](#). In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6450–6470, Albuquerque, New Mexico. Association for Computational Linguistics.
- Z.ai. 2026a. GLM-5.2. <https://docs.bigmodel.cn/cn/guide/models/text/glm-5.2>. Official model documentation. Accessed August 3, 2026.
- Z.ai. 2026b. GLM-5.2 & GLM-5.1 & GLM-5. <https://github.com/zai-org/GLM-5>. Official GLM-5 series repository. Accessed August 3, 2026.
- YanJun Zhao, Ruizhong Qiu, Tianxin Wei, Yuanchen Bei, Zhining Liu, Lingjie Chen, Ismini Lourentzou, Hanghang Tong, and Jingrui He. 2026. [ReContext: Recursive evidence replay as LLM harness for long-context reasoning](#). Preprint, arXiv:2607.02509.

A Attaining the Worst-Case Residual-Map Count

Let $\mathcal{S}$ be a finite nonempty state set, $\mathcal{C}$ an input alphabet, n the sequence length, and $t : \mathcal{C}^n \times \mathcal{S} \rightarrow \mathcal{S}$ a conditional state-update task. For an information sequence $C \in \mathcal{C}^n$, define the residual map and the number of distinct residual maps by

$$\begin{aligned}\phi_C &= (t(C, z))_{z \in \mathcal{S}} \in \mathcal{S}^{\mathcal{S}}, \\ \mathcal{K} &= |\{\phi_C \mid C \in \mathcal{C}^n\}|.\end{aligned}$$

Equivalently, $\phi_C(z) = t(C, z)$. For every finite nonempty state set $\mathcal{S}$, this appendix constructs a task for which $\mathcal{K} = |\mathcal{S}|^{|\mathcal{S}|}$. The corresponding late-order working-memory requirement is

$$\lceil \log_2 \mathcal{K} \rceil = \lceil |\mathcal{S}| \log_2 |\mathcal{S}| \rceil \text{ bits.}$$

Fix an arbitrary finite nonempty state set $\mathcal{S}$. For this $\mathcal{S}$, the construction below chooses $\mathcal{C}$, n , and t and evaluates $\mathcal{K}$ under the definition above. Set $m = |\mathcal{S}|$ and write $b = \log_2 m$. Since every ϕ_C belongs to $\mathcal{S}^{\mathcal{S}}$, $\mathcal{K} \leq m^m$.

Proposition. For every finite nonempty state set $\mathcal{S}$, there exist a finite input alphabet $\mathcal{C}$ and a conditional state-update task such that, with $n = 1$, $\mathcal{K} = m^m$. Every deterministic one-pass processor that is exact on all inputs and reads C before z must therefore retain at least

$$\lceil \log_2 \mathcal{K} \rceil = \lceil m \log_2 m \rceil = \lceil b 2^b \rceil$$

bits of persistent input-dependent state immediately before reading z .

Construction. Set $n = 1$ and choose the input alphabet

$$\mathcal{C} = \{c_f \mid f \in \mathcal{S}^{\mathcal{S}}\}.$$

Thus $|\mathcal{C}| = m^m$. Define one fixed update rule $U : \mathcal{S} \times \mathcal{C} \rightarrow \mathcal{S}$ by $U(s, c_f) = f(s)$, equivalently $U_{c_f} = f$. For every $f \in \mathcal{S}^{\mathcal{S}}$, the sequence $C_f = (c_f)$ belongs to $\mathcal{C}^n$, and its residual map satisfies

$$\phi_{C_f}(z) = t(C_f, z) = U_{c_f}(z) = f(z).$$

Consequently,

$$\{\phi_C \mid C \in \mathcal{C}^n\} = \mathcal{S}^{\mathcal{S}},$$

and

$$\begin{aligned}\mathcal{K} &= |\{\phi_C \mid C \in \mathcal{C}^n\}| \\ &= |\mathcal{S}^{\mathcal{S}}| = m^m.\end{aligned}$$

Late-order cut. Consider the processor configuration after C_f and immediately before z is read. If two distinct functions $f \neq g$ produced the same configuration, some $z \in \mathcal{S}$ would satisfy $f(z) \neq g(z)$. Starting from the shared configuration, the processor would produce the same output after reading the identical suffix z , contradicting exactness. Thus the cut admits at least m^m distinct configurations. At this cut, a processor attains the bound by storing the identity of f in one of m^m states and applying the fixed lookup rule when z arrives. Under the accounting convention above, the exact cut-state requirement for this family is $\lceil \log_2(m^m) \rceil$ bits.

Condition-first comparison. For the order (z, c_f) , a single $\mathcal{S}$ -valued register suffices: initialize it with z and replace its value by $f(z)$ when c_f arrives. At the cut after z and immediately before c_f , this implementation has m possible input-dependent configurations. This is optimal. Let $\iota = \text{id}_{\mathcal{S}}$, so $c_{\iota} \in \mathcal{C}$. If two distinct values $z, z' \in \mathcal{S}$ produced the same configuration at this cut, then reading the identical suffix c_{ι} would force the same output from both configurations. Exactness instead requires the respective outputs $\iota(z) = z$ and $\iota(z') = z'$. Hence the cut admits at least m configurations, and its exact cut-state requirement is $\lceil \log_2 m \rceil = \lceil b \rceil$ bits. When $m = 1$, each cut has one configuration and therefore requires zero bits. As a concrete check, $m = 4$ gives $\mathcal{K} = 4^4 = 256$: the late-order cut requires eight bits, whereas the condition-first cut requires two bits.

Scope. This finite-state existence construction applies to the stated task family and to deterministic, exact, one-pass computation. It is an adversarial worst-case construction whose alphabet and fixed transition table realize all self-maps of $\mathcal{S}$. The processor cannot reread C , and every auxiliary writable store is included in the counted configuration. Restricted update families may realize fewer residual maps, yielding a smaller lower bound from this residual-map argument.

Causal Transformers are Causal State Update Processors. Transformers use kv caches that may expand as context length grows. Modern transformers may include more complex memory structures like latent kv, shared kv or linear kv. Despite these, causal transformers with a finite maximal context length and finite precisions are indeed causal state update processors defined in section 3.1.

Define the "working state" as to contain the current position, all layerwise key-value entries, and any other persistent input-dependent inference buffers. Causal inference computes the new token representation layer by layer and appends the corresponding key-value entries. Thus the next configuration is a fixed function of the preceding configuration and the new token. The maximum context length and finite precision make the configuration space finite. Per-token or other forms of caches therefore do not violate the causal state update abstraction.

B Scoring and Run Qualifications

Section 4.1 gives the common model, dataset, condition, and scoring setup. This appendix specifies scorer edge cases, qualifications of the reported runs, and the record-selection rules used to compute the reported cells.

B.1 Scorer Edge Cases

GraphWalks. The evaluator extracts the terminal line `Final Answer: [...]` and compares the parsed node set with the gold set. A valid empty prediction is scored normally: two empty sets have EM and F1 equal to one, whereas one empty and one nonempty set have EM and F1 equal to zero. An absent response or a missing, malformed, or nonterminal answer receives zero. For Question First, we conservatively treat a null or unknown termination marker as length-limited. In the evaluated records, 121 have a null marker, none have an unknown marker, and 60 are natively marked as length-limited, for 181 length-limited outputs in total (160 BFS and 21 Parents). One additional stopped BFS output lacks the required terminal answer syntax. All 182 outputs receive zero.

MRCRv2 8-needle. Each example supplies a random prefix that must begin both the prediction and reference. The scorer validates and removes this prefix before computing EM and the ratio returned by Python’s `difflib.SequenceMatcher`. EM is one when the remaining strings match exactly and zero otherwise. The paper labels the ratio *Seq.*; it is distinct from GraphWalks set F1. A missing or invalid prefix gives zero for both metrics. Some prompts trigger provider content filters; filtered outputs remain in the denominator and receive zero.

NUB-1M. We maintain a reference answer for each problem and use DeepSeek V4 Pro to com-

pare the extracted solver answer with that reference. Solver identity and feedback order are excluded from the judge prompt. The judge returns a Boolean correctness decision and a short rationale; we use the Boolean as the binary score.

We also manually reviewed the judged outputs and found no errors.

C Prompt and State Templates

This appendix shows the fixed serializer used to construct T and records the dataset-specific prompt details for experiments. We retain the compact condition notation $[T, x]$ for TRACE AS STATE and $[x, T]$ for TRACE APPEND; the paragraphs below give the exact insertion boundaries and fixed interface text suppressed by that notation.

C.1 Reusable Trace Block

Let $r = (r_1, \dots, r_{n_{tr}})$ denote the selected first-pass reasoning traces in source order, and let $P_{\mathcal{D}}$ denote the dataset-specific trace-block preamble containing its description and warning. Table 4 shows their fixed text form.

Here $P_{\mathcal{D}}$ is part of T , rather than part of the original task prompt x ; its exact value for each dataset is given verbatim below. Separately returned visible answers remain outside T . The five source reasoning fields are taken in repeat-index order. When applying the 50,000-character cap, any reasoning field longer than 50,000 characters is replaced by its first 50,000 characters followed by `...`; block formatting then strips leading and trailing whitespace.

C.2 Model-Specific Details

Qwen 3.7 Max reasoning instruction. For every evaluated Qwen 3.7 Max condition, we use the following fixed `xhigh` instruction recommended for reasoning scenarios in the official Qwen 3.7 Max evaluation guidance available when the experiments were run:

Reasoning effort is set to `xhigh`. Please think carefully through the task, validate key assumptions, consider plausible alternatives, and prioritize correctness, consistency, and clarity in the final answer.

Here `xhigh` names the recommended prompt text rather than a provider-side `reasoning_effort` value. For MRCRv2, the runner prepends the instruction to the system

Table 4: The fixed serialization used to construct the reusable trace block.

Reasoning traces r	$T = \pi(r)$
$r = (r_1, r_2, \dots, r_{n_{tr}})$	$P_{\mathcal{D}}$ (<i>dataset-specific prompt</i>) <code><first_run_reasoning_traces></code> <code>[Trace 1]</code> r_1 <code>[Trace 2]</code> r_2 $\vdots$ <code>[Trace n_{tr}]</code> $r_{n_{tr}}$ <code></first_run_reasoning_traces></code>

message. The GraphWalks and NUB-1M runners prepend it to their single user message. Within each dataset, the delivery format is fixed across the evaluated conditions and does not change the relative placement of T and the long context.

Version of Deepseek V4 Pro. There are two models both called Deepseek V4 Pro released on 2026-4-24 and 2026-8-13 respectively. We use the 2026-4-24 version for our experiments.

C.3 Dataset-Specific Details

GraphWalks. For GraphWalks, $P_{\mathcal{D}}$ is the following exact text:

Below are selected reasoning traces or trace tail windows from independent first attempts on the same graph problem. They may contain mistakes. Use them only as scratchpad hints, and verify against the graph.

We also use a system prompt:

You solve directed-graph algorithm problems. Use only the graph and operation in the user message. Return exactly one visible line in this format: Final Answer: [node1, node2]. Use [] for the empty set. Do not include any text before or after that line.

The runner also appends this exact answer-format instruction to the user message:

Return exactly one line in this format: Final Answer: [node1, node2]. Use [] for the empty set.

For $[T, x]$, the complete trace block precedes the complete released GraphWalks prompt, and the answer-format instruction follows that prompt.

For $[x, T]$, the runner splits the released prompt immediately before its last `Operation:` block: the graph instructions and edge list come first, then the identical trace block, then the final operation and answer-format instruction.

This formatting instruction guides the model to output its answer in a parse-able way.

MRCRv2. For MRCRv2, $P_{\mathcal{D}}$ is the following exact text:

Below are reasoning traces from independent first attempts on the same MRCR problem. They may contain mistakes. Use them only as scratchpad hints; verify against the conversation and final request. Do not copy any trace text into the visible answer.

D GraphWalks Difficulty Profiles

As an exploratory diagnostic, we examine whether the descriptive timing gap varies with two observable GraphWalks properties. BFS specifies a requested traversal depth d , and Parents has a gold parent-set size k . These variables summarize aspects of traversal and aggregation demand, but realized difficulty can also depend on frontier size, early termination, and graph structure. The analysis and highlighted ranges were developed after inspecting outcomes; they are hypothesis-generating rather than confirmatory tests of an interaction, threshold, capacity limit, or mechanism.

We pool adjacent low-support values into BFS bins 1–2, 3–4, 5–6, 7–8, and 9–10, and Parents bins 0, 1, 2, 3, 4–5, and ≥ 6 . Their problem counts are (17, 25, 21, 15, 22) and (13, 17, 25, 20, 13, 12), respectively. For feedback timing $p \in \{\text{pre}, \text{post}\}$ and bin b , we report the gain over the common

first-pass baseline,

$$\Delta_p(b) = \text{Score}_p(b) - \text{Score}_{\text{first}}(b). \quad (6)$$

Each point averages five stored repeats within each problem and then the problems in its bin. The yellow regions are descriptive and outcome-informed; their boundaries are specific to each model, subtask, and metric. They were not selected independently of the displayed scores and do not support confirmatory inference.

The prefix-minus-postfix exact-match gap in Figure 3 is concentrated in particular bins. DeepSeek V4 Pro has its largest BFS timing gaps at $d = 3$ – 6 , and Qwen 3.7 Max has a smaller concentration in the same range. GLM-5.2 has positive gaps through $d = 8$ and a reversal in the final pooled bin. The Parents profiles differ. TRACE AS STATE is never below TRACE APPEND in the plotted exact-match bins, with a tie for Qwen 3.7 Max at $k = 0$. Qwen’s separation grows with k ; DeepSeek V4 Pro and GLM-5.2 are nonmonotonic but have large gaps in selected middle or high- k bins.

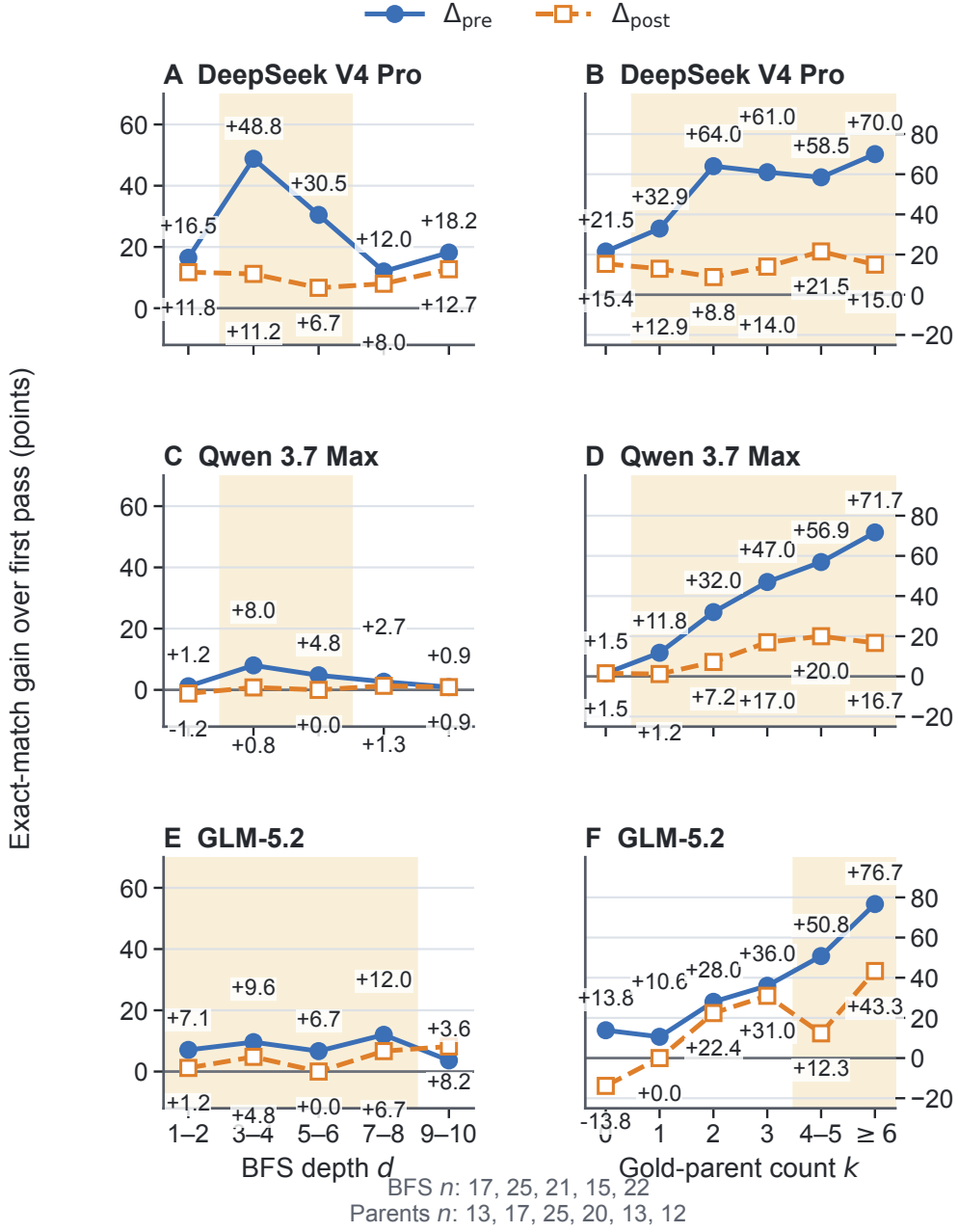


Figure 3: GraphWalks exact-match gains over the first pass by BFS depth d (left) and gold-parent count k (right). The blue and orange curves report Δ_{pre} and Δ_{post} . Bin supports appear below the panels. Yellow shading marks descriptive, outcome-informed ranges of larger separation, with boundaries chosen separately for each model and subtask.

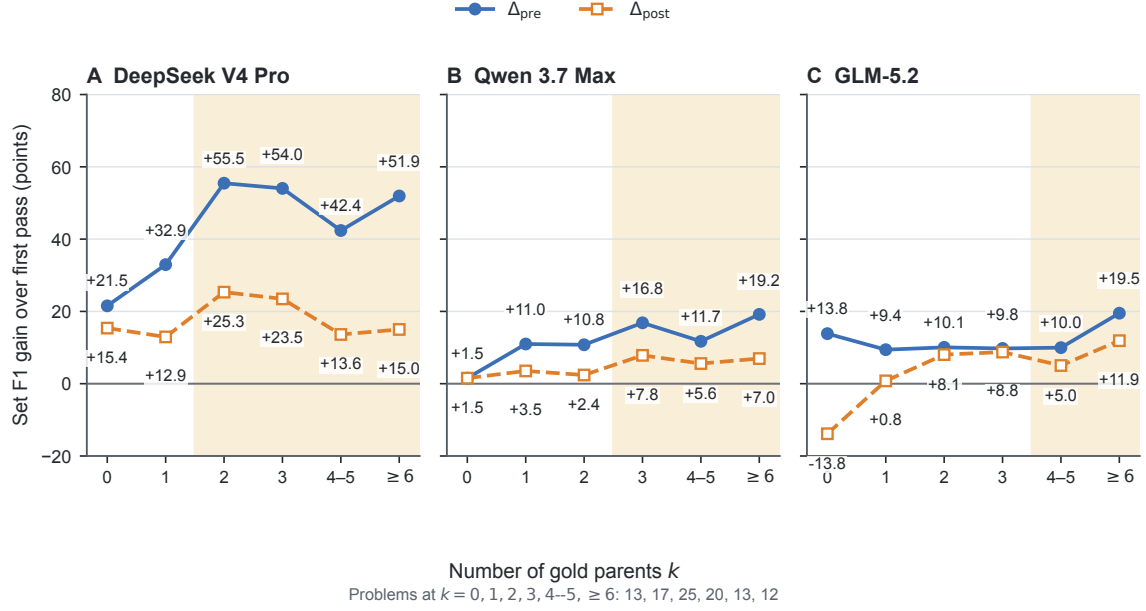
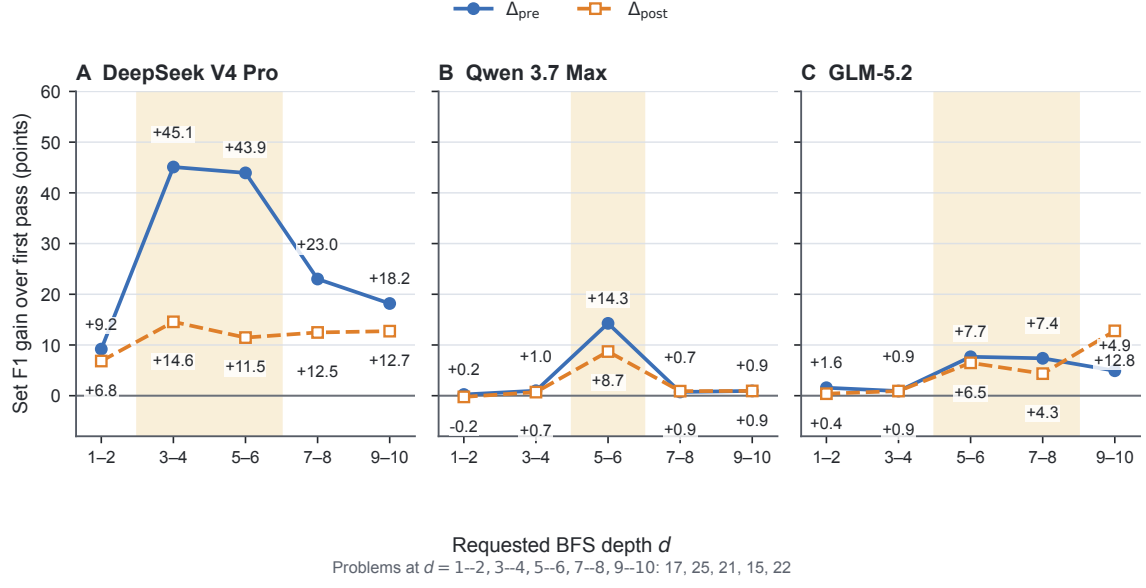


Figure 4: Set F1 gains for the same BFS (top) and Parents (bottom) stratifications. Set F1 records partial overlap between predicted and gold node sets. Yellow shading again marks descriptive, outcome-informed ranges selected separately for each model, subtask, and metric.

The binned set F1 means preserve the strong DeepSeek V4 Pro BFS pattern, while Qwen 3.7 Max and GLM-5.2 show smaller or more localized separation. Parents generally favors TRACE AS STATE, with substantial variation in the size of the gap. These profiles document heterogeneity but do not establish why it occurs or that depth or parent-set size mediates trace reuse. A confirmatory follow-up would define bins or continuous contrasts before observing timing outcomes and evaluate them on held-out problems.

E Confidence Intervals

This appendix reports paired uncertainty for the principal same- T timing contrast and cell-wise uncertainty for the broader results and ablations. Table 5 reports the mean paired TRACE AS STATE minus TRACE APPEND difference and an unadjusted 95% percentile interval after averaging the five repeats within each problem. The intervals lie fully above zero for 20 of 24 cells. The four intervals that cross zero are DeepSeek V4 Pro MRCRv2 512K EM, Qwen 3.7 Max GraphWalks BFS F1, and GLM-5.2 GraphWalks BFS EM and F1.

Figures 5 and 6 report cell means with 95% percentile intervals from the same problem-cluster bootstrap. Repeats are averaged within each problem.

Figure 5 covers the strict same- T GraphWalks and MRCRv2 comparisons in Table 2, together with the supporting NUB-1M results. The NUB-1M intervals are wider, consistent with its 20 evaluated questions. Figure 6 covers the GraphWalks control ablation. The same bootstrap procedure produces the shaded trace-count intervals in Figure 2.

Table 5: Paired problem-cluster uncertainty for the strict same- T timing contrast. Δ is Trace as State minus Trace Append in percentage points; brackets give unadjusted 95% percentile intervals over problems after five-repeat averaging ($n = 100$ per cell; 20,000 resamples; seed 0). GW is GraphWalks 256K and MR is MRCRv2 8-needle. Intervals wholly above zero are bold.

Model	Cell	Δ [95% CI]
DeepSeek V4 Pro	GW BFS EM	+17.00 [+11.00 , +23.40]
	GW BFS F1	+17.63 [+11.65 , +23.97]
	GW Par. EM	+38.80 [+31.60 , +46.20]
	GW Par. F1	+26.02 [+20.52 , +31.84]
	MR 256K EM	+10.20 [+4.80 , +15.80]
	MR 256K Seq.	+9.11 [+4.77 , +13.93]
	MR 512K EM	+3.20 [-2.60, +9.40]
	MR 512K Seq.	+6.35 [+1.69 , +11.36]
Qwen 3.7 Max	GW BFS EM	+3.40 [+1.00 , +6.40]
	GW BFS F1	+1.29 [-0.56, +3.90]
	GW Par. EM	+25.40 [+17.60 , +33.60]
	GW Par. F1	+7.43 [+4.63 , +10.58]
	MR 256K EM	+4.40 [+1.40 , +8.00]
	MR 256K Seq.	+4.19 [+1.41 , +7.53]
	MR 512K EM	+6.40 [+1.80 , +11.40]
	MR 512K Seq.	+5.11 [+0.67 , +9.84]
GLM-5.2	GW BFS EM	+3.40 [-1.00, +7.80]
	GW BFS F1	-0.83 [-3.94, +1.62]
	GW Par. EM	+16.80 [+10.40 , +23.80]
	GW Par. F1	+7.32 [+3.93 , +11.30]
	MR 256K EM	+21.20 [+14.40 , +28.20]
	MR 256K Seq.	+13.56 [+8.33 , +19.34]
	MR 512K EM	+10.80 [+3.40 , +18.20]
	MR 512K Seq.	+10.20 [+4.65 , +16.08]

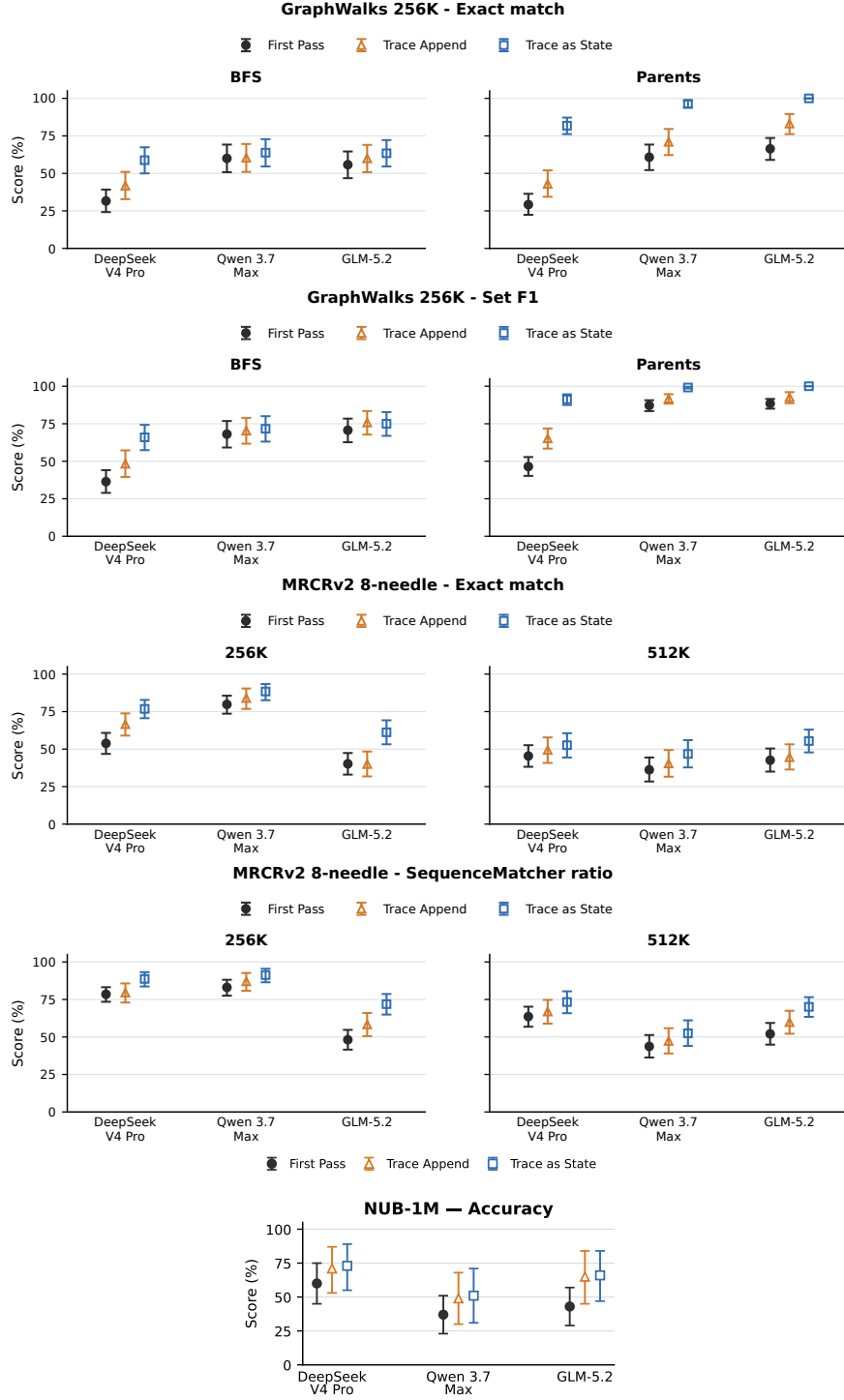


Figure 5: Uncertainty for the strict same- T comparisons. The top two rows show GraphWalks 256K exact match and set F1, split into BFS and Parents; the bottom two wide rows show MRCRv2 8-needle exact match and SequenceMatcher ratio, split into 256K and 512K bins. The final panel shows NUB-1M accuracy. Points are means; bars are 95% percentile intervals after averaging 5 repeats within each problem.

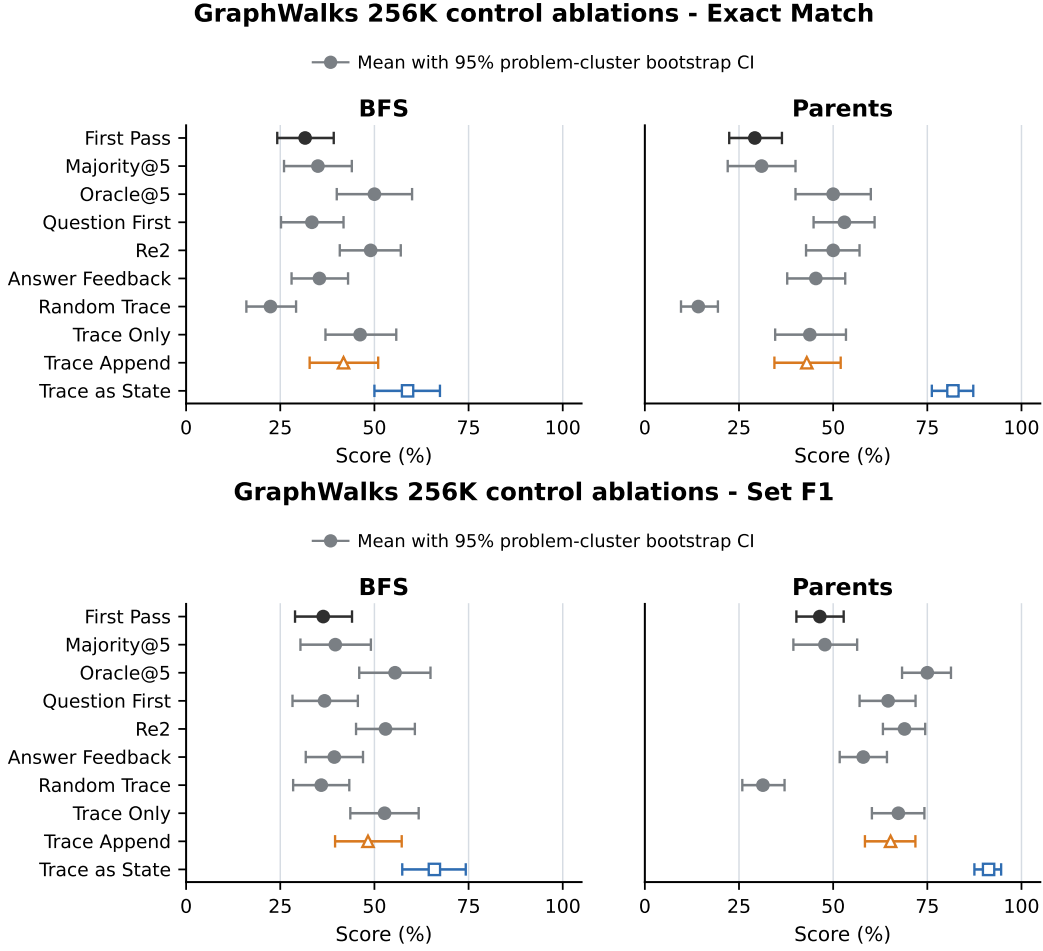


Figure 6: Uncertainty for DeepSeek V4 Pro GraphWalks 256K control comparisons in exact match and set F1. Points are means; bars are 95% percentile intervals from 20,000 problem-cluster bootstrap resamples (seed 0). Majority@5 retains answer elements occurring in at least three repeat-level prediction sets, and Oracle@5 reports the retrospective maximum evaluator score among the five outputs. Other conditions average five planned repeats within each problem and assign zero to missing or invalid outputs.

F Token Usage

Table 6 reports token counts returned by provider APIs for the solver calls underlying the main results.

For each retained response with provider-reported usage, we report cached input, missed input, and output tokens. Missed input is total input minus cached input. Reasoning tokens are included in the output.

First Pass is the shared source pass used to construct T . For TRACE AS STATE and TRACE APPEND, we also report the *Total tokens* columns that add the corresponding First Pass counts component-wise.

Table 6: Provider-reported token usage for main-result inference, in millions. *Tokens* reports the pass named in each row. For TRACE APPEND and TRACE AS STATE, *Total tokens* adds First Pass component-wise; those columns are blank for First Pass itself. Missed input is total input minus cached input.

Benchmark	Condition	Tokens			Total tokens		
		Cached input	Missed input	Output	Cached input	Missed input	Output
<i>DeepSeek V4 Pro</i> GraphWalks 256K	First Pass	206.766	51.092	62.507			
	TRACE APPEND	271.017	68.554	35.231	477.783	119.646	97.738
	TRACE AS STATE	277.074	62.501	27.975	483.840	113.593	90.482
MRCRv2 256K	First Pass	50.817	46.961	1.149			
	TRACE APPEND	79.982	20.820	0.690	130.799	67.782	1.839
	TRACE AS STATE	51.875	50.051	1.355	102.692	97.012	2.504
MRCRv2 512K	First Pass	155.283	38.195	0.989			
	TRACE APPEND	178.675	18.526	0.702	333.958	56.721	1.691
	TRACE AS STATE	157.261	39.940	1.241	312.544	78.135	2.230
NUB-1M Season 2	First Pass	31.651	8.418	0.817			
	TRACE APPEND	39.589	4.460	0.184	71.240	12.878	1.001
	TRACE AS STATE	27.418	16.637	0.398	59.069	25.055	1.215
<i>Qwen 3.7 Max</i> GraphWalks 256K	First Pass	260.213	97.483	5.402			
	TRACE APPEND	264.906	117.866	2.865	525.119	215.350	8.267
	TRACE AS STATE	271.742	111.030	3.615	531.955	208.514	9.017
MRCRv2 256K	First Pass	38.337	63.495	2.111			
	TRACE APPEND	60.227	51.195	0.989	98.564	114.690	3.100
	TRACE AS STATE	62.213	49.209	1.397	100.550	112.704	3.508
MRCRv2 512K [†]	First Pass	134.607	63.738	2.263			
	TRACE APPEND	154.827	53.489	1.211	289.434	117.226	3.474
	TRACE AS STATE	153.971	54.345	1.724	288.577	118.083	3.988
NUB-1M Season 2	First Pass	34.253	7.547	0.489			
	TRACE APPEND	38.184	5.773	0.253	72.437	13.320	0.742
	TRACE AS STATE	32.414	11.549	0.419	66.667	19.096	0.908
<i>GLM-5.2</i> GraphWalks 256K	First Pass	134.748	150.760	19.463			
	TRACE APPEND	55.612	265.396	9.127	190.361	416.156	28.591
	TRACE AS STATE	64.390	256.618	16.491	199.138	407.377	35.954
MRCRv2 256K	First Pass	73.782	24.718	1.891			
	TRACE APPEND	82.413	20.940	2.518	156.195	45.658	4.409
	TRACE AS STATE	82.464	20.888	1.516	156.246	45.606	3.407
MRCRv2 512K	First Pass	151.247	42.262	1.441			
	TRACE APPEND	157.740	40.508	2.002	308.987	82.770	3.443
	TRACE AS STATE	157.458	40.790	1.648	308.705	83.052	3.089
NUB-1M Season 2	First Pass	1.272	41.123	0.995			
	TRACE APPEND	3.840	43.087	0.484	5.111	84.209	1.478
	TRACE AS STATE	0.457	46.475	0.856	1.728	87.598	1.850